第三章 系统总体设计

本章围绕“基于嵌入式平台的目标检测系统研究”的总体实现目标，对无人机目标检测系统的功能需求、性能需求、部署约束和模块架构进行设计。系统以 RK3588 嵌入式平台为主要部署对象，利用板载神经网络处理单元（NPU）完成目标检测模型推理，并结合实时视频采集、检测结果后处理、轻量级目标跟踪、动态感兴趣区域（ROI）以及多推理上下文（context）并行推理策略，实现面向边缘端场景的目标检测与显示。

与单纯在 PC 端运行目标检测算法不同，嵌入式部署需要同时关注检测精度、实时性、资源占用和系统稳定性。本文的系统设计并不追求在所有公开数据集上获得最高精度，而是以“模型能够在 RK3588 平台稳定运行，并在实时视频流中输出可观察、可记录、可复现实验结果”为核心目标。因此，本章将系统划分为模型移植、视频输入、NPU 推理、后处理与显示、性能记录和多线程优化等模块，并为后续系统实现与实验分析提供设计依据。

3.1 系统需求分析

3.1.1 功能需求

本文系统面向嵌入式平台上的小目标检测应用，主要完成视频输入、目标检测、结果可视化和实验记录等功能。系统需要能够读取本地视频、USB 摄像头或实时流输入，在每帧图像中识别目标位置，并将检测框、类别名称和置信度绘制到输出画面中。为了支撑论文实验，系统还需要保存检测结果、运行帧率、推理耗时和资源占用等数据，使同一段视频能够在不同配置下进行重复测试。

系统的主要功能需求如表 3.1 所示。

表 3.1 系统功能需求

序号	功能模块	需求说明
1	视频输入	支持本地视频文件、USB 摄像头和实时视频流输入，并能够在固定视频上进行重复实验。
2	模型推理	在 RK3588 平台上加载 RKNN 模型，调用 NPU 完成目标检测推理。
3	目标后处理	对模型输出进行解析、置信度筛选和非极大值抑制，得到目标框坐标和置信度。
4	可视化显示	在输出画面中绘制检测框、类别名称和置信度，并通过视频文件或 RTSP 流显示检测结果。
5	性能记录	记录流处理帧率、NPU 推理帧率、端到端延迟、CPU 占用和内存占用等指标。
6	策略配置	支持检测间隔、多线程推理、平滑参数、ROI 策略等运行参数配置，便于实验对比。

3.1.2 性能需求

嵌入式目标检测系统需要在有限算力和功耗条件下保持较高实时性。由于无人机等小目标在画面中可能只占据较少像素，系统既要尽量保证检测召回率，又要避免过高的推理频率导致视频处理阻塞。结合 RK3588 平台的实际部署情况，本文将性能需求分为实时性、稳定性、可复现性和资源约束四个方面。

首先，系统需要保证视频流能够连续输出，避免因推理过慢造成明显卡顿。其次，检测结果需要具有一定稳定性，避免检测框在相邻帧之间出现明显抖动。再次，系统需要能够在相同输入视频和相同参数下得到可复现的实验日志。最后，系统应尽量降低 CPU 端后处理和视频编码的负担，将主要神经网络计算交由 NPU 完成。

表 3.2 给出了本文系统设计中关注的主要性能指标。

表 3.2 系统性能指标

指标名称	含义	设计目标
stream_fps	输出视频流的实际帧率	保证画面连续，尽量接近输入视频帧率。
npu_fps	NPU 完成推理的帧率	反映模型推理吞吐能力，用于比较不同 context 数量。
端到端延迟	输入帧进入系统到输出检测结果的整体延迟（end-to-end latency）	在实时显示场景下尽量降低。
CPU 占用率	视频读取、预处理、后处理和编码造成的 CPU 负载	避免 CPU 成为主要瓶颈。
RSS 内存	程序运行时常驻内存占用	保证长时间运行稳定。
误检与漏检	检测结果中的错误目标和漏掉目标	用于分析模型在公开视频和真实输入中的适用性。

3.1.3 部署约束

本文系统的部署对象为 RK3588 嵌入式平台。该平台具备多核 CPU、GPU 和 NPU 等计算资源，但在实际系统中，视频采集、图像预处理、NPU 推理、后处理和视频输出之间仍然存在明显的调度关系。如果各模块串行执行，任一环节耗时过长都会影响实时显示效果。因此，系统需要采用多线程流水线结构，将视频采集、模型推理和结果输出进行解耦。

此外，模型格式也是部署约束之一。训练阶段通常得到 PyTorch 权重文件或 ONNX 模型，而 RK3588 的 NPU 推理需要使用 RKNN 格式模型。因此，系统需要建立从训练模型到 RKNN 模型的转换流程，并在板端验证模型输入输出张量是否与后处理代码匹配。本文当前稳定运行的主路径采用浮点 RKNN 模型，INT8 量化优化和 RGA 硬件预处理作为后续扩展方向，不在本章中作为已完成主线进行描述。

3.2 系统总体架构

本文系统采用“视频输入层、预处理层、NPU 推理层、后处理层、策略控制层和结果输出层”的分层架构，如图 3.1 所示。视频输入层负责从本地文件、摄像头或 RTSP 输入中获取图像帧；预处理层负责完成图像缩放、颜色空间转换和输入张量

组织；NPU 推理层负责调用 RKNN Runtime 完成神经网络前向计算；后处理层负责解析模型输出并生成检测框；策略控制层负责检测间隔、结果复用、动态 ROI、框平滑和轻量跟踪；结果输出层负责视频显示、RTSP 推流和日志保存。

图 3.1 系统总体架构图

在系统运行过程中，视频帧首先进入采集线程。采集线程不直接等待 NPU 推理完成，而是将帧数据写入有限长度队列。推理线程从队列中取出图像帧，按照当前策略决定是否执行完整 NPU 检测。如果当前帧需要检测，则调用 RKNN 模型并更新检测结果；如果当前帧不需要检测，则根据上一帧检测框和轻量跟踪结果进行复用。输出线程负责将最新检测结果绘制到画面中，并通过视频文件或 RTSP 服务输出。

这种设计将视频输入、模型推理和显示输出解耦，避免单个慢速模块阻塞整个系统。对于固定视频实验，系统可以保存带框输出视频和逐帧检测结果；对于实时演示，系统可以通过 RTSP 地址在电脑端查看检测画面。固定视频实验保证可复现性，实时视频流验证实际部署效果，两者共同支撑后续实验分析。

3.3 模型移植与检测模块设计

3.3.1 模型移植流程

本文的目标检测模型首先在 PC 或 WSL 环境中完成训练和导出，然后转换为 RK3588 平台可加载的 RKNN 模型。整体流程如图 3.2 所示。

图 3.2 模型移植流程图

在模型移植过程中，关键问题不是简单完成格式转换，而是保证模型输出与板端后处理逻辑一致。本文当前使用的稳定模型为单类别 drone 检测模型，板端输出张量形式为 [1, 5, 8400]，其中 5 个通道对应目标框参数和置信度信息。后处理模块需要将模型输出解析为边界框中心点、宽高和置信度，再映射回原始图像坐标系。

对于无人机检测任务，类别数量较少，后处理重点主要集中在置信度阈值、坐标还原和非极大值抑制上。由于目标在远距离情况下通常较小，阈值设置过高容易造成漏检，阈值设置过低则会增加误检。因此，系统将置信度阈值和 NMS 阈值设计为可配置参数，在不同视频场景中进行调试和记录。

3.3.2 检测结果解析

检测模块的输入为预处理后的图像张量，输出为若干候选框。每个候选框包含左上角坐标、右下角坐标、类别编号和置信度。由于当前模型为单类别无人机检测模型，因此类别编号统一映射为 drone。在结果可视化时，系统将类别名称和置信度绘制在检测框附近，便于观察模型输出。

检测结果解析过程包括以下步骤：

1. 读取 RKNN 输出张量，并按照模型输出格式转换为候选框；
2. 根据置信度阈值过滤低置信度候选框；
3. 将网络输入尺寸下的坐标还原到原始图像尺寸；
4. 对重叠候选框执行非极大值抑制；
5. 输出最终检测框，用于绘制、日志保存和后续跟踪。

从设计角度看，检测后处理模块需要保持独立，避免与视频输入、RTSP 输出和多线程调度强耦合。这样做的好处是：当后续更换模型结构、输入尺寸或类别数量时，只需要调整模型加载和后处理逻辑，而不需要重写整个视频处理系统。

3.4 实时视频处理流水线设计

3.4.1 多线程流水线结构

为了提高实时性，本文系统采用多线程流水线设计。系统主要包含采集线程、推理线程和输出线程。采集线程负责从视频源读取帧；推理线程负责执行预处理、NPU 推理和后处理；输出线程负责绘制结果并编码输出视频流。多线程结构如图 3.3 所示。

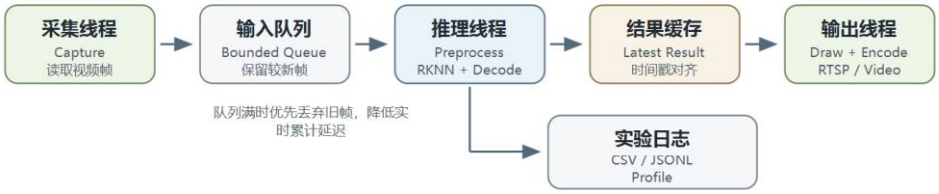

图 3.3 实时视频处理流水线

在该结构中，输入队列采用有限长度设计。当推理速度低于视频输入速度时，系统优先丢弃过旧帧，保留较新的输入帧。这样可以降低实时画面的累计延迟，避免系统为了处理历史帧而导致显示严重滞后。对于实时演示场景，低延迟通常比完整处理每一帧更重要；对于固定视频评测场景，则可以通过离线模式保存完整结果。

3.4.2 实时输出方式

本文系统支持两类输出方式。第一类是固定视频输出，即将检测框绘制到视频文件中，并保存逐帧检测结果。这种方式适合论文实验，因为同一输入视频可以在不同参数配置下重复运行。第二类是实时 RTSP 输出，即将带框画面编码为视频流，电脑端可以通过播放器打开 RTSP 地址进行观看。这种方式适合系统演示和板端联调。

固定视频和实时 RTSP 的定位不同。固定视频用于严谨对比，能够得到稳定的帧率、延迟和检测结果统计；实时 RTSP 用于验证系统是否能在实际部署中持续运行。本文后续实验中，固定视频作为主要对比入口，实时 RTSP 作为最终演示入口。

3.5 动态 ROI 与轻量跟踪策略设计

3.5.1 检测间隔策略

在嵌入式实时检测中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升高或视频卡顿。为此，本文设计了检测间隔策略，即每隔 N 帧执行一次完整检测，中间帧复用上一帧检测结果或通过轻量跟踪进行更新。该策略通过参数 detect_every_n 控制。

设当前帧编号为 k，检测间隔为 N，则是否执行完整检测可表示为：

$$\text{Infer}(k)=1,\ k \bmod N = 0;\ \text{Infer}(k)=0,\ k \bmod N \neq 0 \tag{3.1}$$

当 Infer(k)=1 时，系统执行完整 NPU 检测；当 Infer(k)=0 时，系统不调用 NPU，而是根据上一帧结果和跟踪策略生成当前帧的显示框。该方法能够降低 NPU 调用频率，从而改善输出画面的连续性和系统延迟。

3.5.2 动态 ROI 策略

无人机目标在画面中通常较小，并且在相邻帧之间位置变化具有连续性。基于这一特点，系统设计了动态 ROI 策略：当上一帧存在稳定检测结果时，系统可以围绕目标位置生成局部区域，用于后续帧的跟踪、结果复用或局部分析。动态 ROI 的作用不是替代模型检测，而是在检测间隔期间提供目标区域约束，减少检测框跳变。

动态 ROI 的设计需要考虑两个问题。第一，ROI 区域不能过小，否则目标快速运动时容易丢失；第二，ROI 区域不能过大，否则对抑制背景干扰的作用会减弱。因此，系统在检测框周围保留一定扩展边界，并结合图像边界进行裁剪，保证 ROI 始终位于有效图像范围内。

3.5.3 轻量跟踪与框平滑

在实时显示中，即使模型能够正确识别目标，检测框也可能因为输入画面噪声、目标尺寸较小和模型置信度波动而产生抖动。为改善显示效果，本文系统设计了轻量级跟踪和平滑策略。跟踪策略用于在未执行完整检测的帧中估计目标位置；框平滑策略用于减少相邻检测框之间的突变。

框平滑采用指数平滑思想，将当前检测框与上一帧显示框进行加权融合：

$$B_smooth(k)=alpha\ B_detect(k)+(1-alpha)B_smooth(k-1) \tag{3.2}$$

其中，B_detect(k) 表示当前帧检测框，B_smooth(k-1) 表示上一帧平滑后的检测框，alpha 为平滑系数。alpha 越大，检测框越接近当前检测结果；alpha 越小，检测框越稳定但响应速度越慢。本文在实际测试中将平滑参数设计为可配置项，用于在稳定性和响应速度之间取得平衡。

3.6 多 context NPU 并行推理设计

3.6.1 多 context 设计思想

RK3588 平台的 NPU 具备并行计算能力，但在单线程、单 context 调用方式下，视频预处理、输入设置、NPU 执行和输出读取往往串行发生，无法充分重叠不同帧之间的处理过程。为了提高 NPU 使用效率，本文设计了多 context 并行推理结构，即创建多个 RKNN context，并由多个推理工作线程并行处理不同帧。

多 context 并行推理结构如图 3.4 所示。

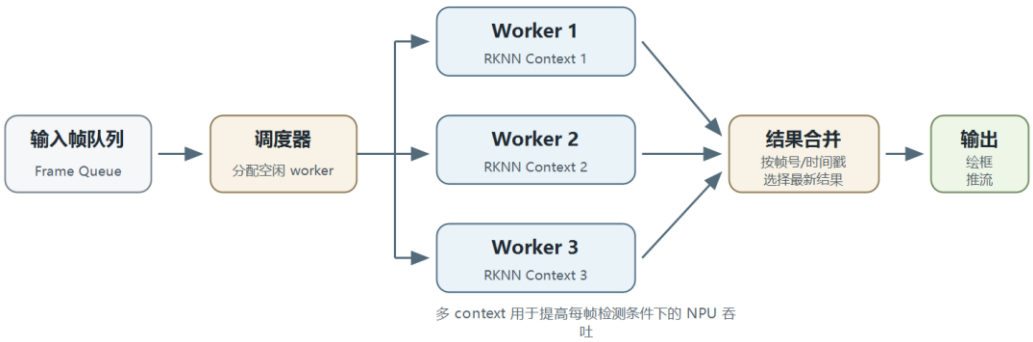

图 3.4 多 context NPU 并行推理结构

在该结构中，每个推理线程拥有独立的 RKNN context，线程之间不共享模型运行状态。输入帧由调度器分配给空闲 worker，推理完成后将结果写入结果队列。输出端根据时间戳或帧编号选择最新结果进行绘制，避免较旧推理结果覆盖较新画面。

3.6.2 多 context 与检测间隔的关系

检测间隔策略和多 context 策略解决的是不同问题。检测间隔通过减少 NPU 调用次数降低系统负载，适合实时演示中对低延迟和稳定显示的需求；多 context 策略通过并行化提高 NPU 推理吞吐，适合分析 RK3588 的硬件能力和每帧检测场景。

在实际设计中，两种策略可以组合使用。例如，当 detect_every_n=2 时，系统每两帧执行一次完整检测，可以获得较好的显示稳定性；当 detect_every_n=1 且启用两个 context 时，系统可以测试每帧检测条件下的最高 NPU 吞吐能力。本文后续实验将比较单 context、双 context 和多 context 的差异，分析不同配置对 stream_fps、npu_fps 和端到端延迟的影响。

3.6.3 阶段耗时分析设计

为了定位系统瓶颈，本文在推理模块中设计了阶段耗时记录功能。每次推理过程被划分为图像准备、输入设置、NPU 运行、输出读取、检测解码和资源释放等阶段。通过记录各阶段耗时，可以判断瓶颈来自 NPU 计算、输入数据传输、后处理还是视频输出。

阶段耗时分析的意义在于避免盲目优化。例如，如果后处理耗时很低，而输入设置和 NPU 运行耗时占主要比例，则继续优化 NMS 或绘框代码对整体实时性提升有限；此时更值得关注输入格式、数据传输、多 context 调度或模型结构本身。本文后续实验将利用该设计对 RK3588 板端运行过程进行量化分析。

3.7 本章小结

本章完成了基于 RK3588 嵌入式平台的目标检测系统总体设计。首先，从功能需求、性能需求和部署约束三个方面分析了系统需要解决的问题；随后，提出了由视频输入、预处理、NPU 推理、后处理、策略控制和结果输出组成的分层架构；接着，设计了模型移植流程、实时视频处理流水线、动态 ROI 与轻量跟踪策略；最后，给出了面向 NPU 性能优化的多 context 并行推理设计和阶段耗时分析方法。

通过上述设计，系统能够在保证原有检测功能可运行的基础上，进一步支持固定视频评测、实时 RTSP 演示和多线程 NPU 性能对比。下一章将围绕各模块的具体实现展开，包括 RKNN 模型加载、视频流处理、检测后处理、RTSP 输出、多线程调度以及实验日志记录等内容。